TÊN CHỦ ĐỀ BÁO CÁO

TRƯỜNG ĐẠI HỌC BÁCH KHOA — ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Báo cáo Tuần — Môn: Lập trình Đa nền tảng

---

1. Nguyễn Văn A – MSSV: xxxxxxxxxx
2. Trần Thị B – MSSV: xxxxxxxxxx

GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

1. Phần I: Cơ Sở Lý Thuyết & Bản Chất Công Nghệ
   - Đặt vấn đề & Mục tiêu
   - Nguyên lý & Cơ chế hoạt động
2. Phần II: Kiến Trúc Giải Pháp & Luồng Dữ Liệu
   - Sơ đồ kiến trúc & Phân tầng
   - Luồng dữ liệu (Data Flow)
3. Phần III: Demo Thực Tế & Mã Nguồn Then Chốt
   - Cấu trúc thư mục & Mã nguồn then chốt

- Kịch bản Unit Test & Kết quả
- Tài liệu tham khảo

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Nguyễn Văn A	Nghiên cứu lý thuyết, thiết kế kiến trúc, viết UseCase & Unit Test	50%
2	Trần Thị B	Thiết lập DI, hiện thực tầng Data/API, xây dựng giao diện UI & BLoC	50%

Phần I: Cơ Sở Lý Thuyết & Bản Chất Công Nghệ

Đặt vấn đề & Mục tiêu

- **Vấn đề thực tế:**
  - Nêu ngắn gọn bài toán kỹ thuật cần giải quyết trong tuần.
  - Hạn chế của cách làm truyền thống nếu không áp dụng kiến trúc / kỹ thuật này.
- **Mục tiêu kỹ thuật:**
  - Nắm vững nguyên lý và cơ chế hoạt động ngầm (under-the-hood).
  - Phân tách rõ ràng trách nhiệm giữa các module.
  - Đảm bảo khả năng bảo trì, mở rộng và viết Unit Test độc lập.

Nguyên lý & Cơ chế hoạt động

Khái niệm cốt lõi:

- Giải thích khái niệm 1.
- Giải thích khái niệm 2.
- Cơ chế vòng đời / luồng điều khiển.

Điểm mấu chốt: Nhấn mạnh bản chất kỹ thuật để phục vụ phần hỏi đáp phản biện (Q&A).

Sơ đồ nguyên lý

Chèn sơ đồ / hình minh họa kiến trúc tại đây

```
#image("path/to/diagram.png",
width: 90%)
```

Phần II: Kiến Trúc Giải Pháp & Luồng Dữ Liệu

Sơ đồ kiến trúc & Phân tầng

- **Tầng 1 (Cấp cao / UI):** Mô tả vai trò và các component chính.
- **Tầng 2 (Trung gian / Nghiệp vụ):** Mô tả luồng xử lý và abstraction.
- **Tầng 3 (Cấp thấp / Dữ liệu):** Mô tả tích hợp API, Database, Local Storage.

Quy tắc phụ thuộc (Dependency Rule)

Tầng bên ngoài phụ thuộc vào tầng bên trong. Tầng bên trong độc lập tuyệt đối với framework và nguồn dữ liệu bên ngoài.

Luồng dữ liệu (Data Flow)

1. **Bước 1 (User Action):** Người dùng tương tác trên Widget / UI.
2. **Bước 2 (State Management):** UI gửi Event đến State Manager (BLoC / Riverpod).
3. **Bước 3 (Domain Execution):** Gọi UseCase xử lý nghiệp vụ độc lập.
4. **Bước 4 (Data Retrieval):** Repository điều phối gọi DataSource (API / Local DB).
5. **Bước 5 (State Emission):** Trả kết quả (Data/Failure), UI cập nhật lại giao diện.

Phần III: Demo Thực Tế & Mã Nguồn Then Chốt

Cấu trúc thư mục & Mã nguồn then chốt

Cấu trúc dự án:

```
lib/
├── core/
├── di/
├── features/
│   ├── data/
│   ├── domain/
│   └── presentation/
└── main.dart
```

Đoạn mã then chốt (Dependency Injection / Logic):

```dart
// Đoạn mã khởi tạo hoặc UseCase chính
final sl = GetIt.instance;
void setupLocator() {
  sl.registerLazySingleton<Repository>(
    () => RepositoryImpl(dataSource: sl()),
  );
}
```

Kịch bản Unit Test & Kết quả

- **Chiến lược kiểm thử:**
  - Sử dụng Mock Object để cô lập module cần kiểm thử.
  - Không phụ thuộc mạng thật (No real network calls), không dựng widget nặng.

Các ca kiểm thử chính (Test Cases)

- **Case 1 (Success):** Mock trả về kết quả hợp lệ ⟶ Output khớp kỳ vọng.
- **Case 2 (Failure):** Mock ném ngoại lệ ⟶ Xử lý lỗi an toàn, không crash app.
- **Kết quả:** Chạy lệnh `flutter test` – 100% test cases pass.

Tài liệu tham khảo

1. Flutter Official Documentation: https://docs.flutter.dev
2. Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
3. Dart Language Tour & Asynchronous Programming Guide: https://dart.dev/guides
4. Mã nguồn dự án và hướng dẫn chạy: https://github.com/your-username/your-repo